

UC MATH 600 MATHEMATICS FOR COMPUTER SCIENCE · MODULE 10 · LECTURE NOTES

Recurrences

How recursive algorithms get their running times.

MODULE OUTCOMES

- Set up a recurrence from a recursive algorithm.
- Solve by expansion and by recursion tree.
- State and apply the master theorem.
- Verify a guess by substitution and induction.
- Recognise when the master theorem does not apply.

1 Setting one up

A RECURRENCE IS A DIRECT TRANSCRIPTION OF WHAT THE ALGORITHM DOES, AND GETTING IT WRONG IS THE USUAL FAILURE

Count three things: how many recursive calls, the size of each subproblem, and the non-recursive work done at this level — and the recurrence writes itself as $T(n) = (\text{calls}) \cdot T(\text{size}) + (\text{work})$.

Merge sort: two recursive calls, each on half the input, plus linear time to merge — $T(n) = 2T(n/2) + \Theta(n)$. Binary search: one call on half, constant work — $T(n) = T(n/2) + \Theta(1)$.

And the base case matters: $T(1) = \Theta(1)$ is nearly always what you want, and omitting it is Module 03's first error arriving in a new setting — a recurrence without a base case determines nothing.

Plus the floors and ceilings are usually ignorable for asymptotic purposes — $T(\lceil n/2 \rceil)$ behaves like $T(n/2)$ — and saying so once explicitly is better than carrying them through every line, which CLRS does and then justifies.

2 Solving by expansion

$$T(n) = 2T(n/2) + n$$

EXPAND

$$= 2[2T(n/4) + n/2] + n = 4T(n/4) + 2n$$

$$= 4[2T(n/8) + n/4] + 2n = 8T(n/8) + 3n$$

...

$$= 2^k T(n/2^k) + kn$$

STOP when $n/2^k = 1$, i.e. when $k = \log_2(n)$

$$= n T(1) + n \log_2(n)$$

$$= \Theta(n \log n)$$

THE RECURSION TREE VIEW, which is the same thing

level 0: one problem of size n , work n

level 1: two of size $n/2$, work n

level 2: four of size $n/4$, work n

...

$\log n$ levels, each doing n work $\rightarrow n \log n$

AND THE TREE VIEW IS THE ONE TO KEEP: it shows WHERE the cost is. If the work per level grows as you descend, the leaves dominate; if it shrinks, the root dominates; if it is constant, every level matters.

The recursion tree shows *where* the cost is — and whether the root, the leaves, or every level equally dominates is exactly what the master theorem's three cases encode (§3). Drawing the tree for three or four recurrences before meeting the theorem makes the theorem obvious rather than arbitrary, which is why this section comes first.

3 The master theorem

for $T(n) = a \cdot T(n/b) + f(n)$, compare $f(n)$ against $n^{\log_b a}$

- $n^{\log_b a}$ is the total work done at the leaves of the recursion tree, and the comparison decides which level dominates — which is precisely §2's tree picture turned into a rule.
- If f is polynomially smaller: $T(n) = \Theta(n^{\log_b a})$ — the leaves dominate. Example: $T(n) = 4T(n/2) + n$ gives $\Theta(n^2)$, since the leaf work grows as you descend.
- If f is the same order: $\Theta(n^{\log_b a} \log n)$ — every level contributes equally, and there are $\log n$ levels. Merge sort is this middle case.
- If f is polynomially larger: $\Theta(f(n))$ — the root dominates and the recursion is cheap. $T(n) = 2T(n/2) + n^2$ is this case.
- The three cases are the three tree shapes — leaves dominate, all levels equal, or root dominates — which is why §2 is worth doing before memorising this. Derive the master theorem from the tree once, and you will never misapply it.

AND IT DOES NOT ALWAYS APPLY, WHICH IS WHAT THE WORD 'POLYNOMIALLY' IS DOING

The comparison has to be polynomial — f must be smaller or larger than $n^{\log_b a}$ by a factor of n to some positive power, not merely smaller or larger, and that qualifier is routinely dropped when the theorem is quoted.

So $T(n) = 2T(n/2) + n \log n$ falls straight into the gap — $n \log n$ is larger than n , and is not polynomially larger (no n^ϵ fits between them) — and the theorem simply says nothing about it.

Which is handled by expansion (§2), giving $\Theta(n \log^2 n)$ in a few lines — so the fallback is always available, and that is why §2 comes first rather than being presented as the slow method.

And the regularity condition attached to case three (that $a \cdot f(n/b) \leq c \cdot f(n)$ for some $c < 1$) is almost always satisfied by the functions you meet, and should be checked once rather than assumed forever.

4 Substitution

- **Guess the answer** — from an expansion, from the master theorem, or from a recurrence you have seen before — and then prove it by induction (Module 03), which is where this method gets its rigour.
- Which makes it the only one of the three methods that is actually a proof: expansion contains an informal 'and so on' that hides an induction, and the master theorem is a proof only once its conditions have been checked (§3's callout).
- And the standard trap: trying to prove $T(n) \leq cn$ fails even when the bound is true, because the induction does not carry — the constants do not work out — and strengthening the hypothesis to $T(n) \leq cn - d$ fixes it.

- **Which is counterintuitive and correct: a stronger claim is sometimes easier to prove by induction, because the hypothesis you get to assume is stronger too — which is the one genuinely counterintuitive fact in this module,** and is worth meeting here rather than in the middle of CSCE 629.
- **And Project 2 asks for three recurrences solved three ways, with the three answers agreeing** and any disagreement tracked down rather than averaged.

AND THIS IS THE DIRECT BRIDGE INTO CSCE 629

Every divide-and-conquer algorithm in that course has its running time established by one of these three methods — usually the master theorem, usually in a single line, usually without comment.

So being fluent here means those lines are readable — and not being fluent means the course's central analytic technique arrives as an assertion you have to take on trust, which is exactly Module 01 §4's attrition mechanism.

Which makes this the clearest case in the whole prerequisite of a specific skill with a specific, nameable payoff, and worth the practice even if the rest of the course felt easy.

And the three methods have a clean division of labour: the recursion tree shows you where the cost is, the master theorem gives you the answer quickly, and substitution proves it — use all three, in that order.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
CLRS, chapter 4 (library copy)	The whole module — the three methods, the master theorem's proof, and the substitution trap of §4 worked.
MIT 6.006, the divide-and-conquer lectures (free)	§§2 and 3 — recursion trees drawn, which is the right way to meet them.
Lehman, Leighton & Meyer, chapter 21 (free)	§§2 and 4 , free — recurrences solved by expansion and by guess-and-verify.
Mitzenmacher & Upfal, the recurrence material	§4 for randomised algorithms — where the recurrences become expectations, which is CSCE 658. Library copy.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Write recurrences** for merge sort, binary search, and Strassen's multiplication.
- 2 **Solve $T(n) = 2T(n/2) + n$ by expansion**, showing every level.
- 3 **Draw the recursion tree** for three recurrences and identify the dominant level.
- 4 **Apply the master theorem** to ten recurrences.
- 5 **Find two where it does not apply**, and say why.
- 6 **Solve one of those by expansion**.
- 7 **Prove a bound by substitution**, with the induction explicit.
- 8 **Hit the $T(n) \leq cn$ trap** deliberately, then fix it by strengthening.
- 9 **Explain why strengthening helped**, in three sentences.
- 10 **Solve three recurrences three ways** and check the answers agree.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What three quantities give you a recurrence?
- 2 Why does the base case matter?
- 3 Solve $T(n) = 2T(n/2) + n$ by expansion.
- 4 What does the recursion tree show that the algebra does not?
- 5 State the master theorem's three cases.
- 6 How do the cases correspond to tree shapes?
- 7 What is 'polynomially' doing, and give a recurrence in the gap.
- 8 Which method is actually a proof, and why are the others not?
- 9 State the substitution trap and its fix.
- 10 Why can a stronger claim be easier to prove?